Cracking My First Binary

Integer Overflow — From Reverse Engineering to Infinite Passwords

Target: izijerry's "int overflow" — crackmes.one

Tools: Ghidra · GDB · Python · Linux Terminal

March 2026

1. Overview

> Challenge: izijerry's "int overflow"
> URL: https://crackmes.one/crackme/6894d8e98fac2855fe6fb3ff
> Platform: Linux x86-64
> Language: C/C++
> Difficulty: 1.4 / 6
> Vulnerability: Integer overflow in atoi() on user-controlled input
> Passwords found: 10 confirmed (infinite series proven)

TL;DR

The program converts user input with atoi() and compares the result to -0x2023e3c2.

Because atoi() returns a signed 32-bit integer, large numbers overflow and wrap around.

The first valid password is:

3755744318

Infinite valid passwords exist:

password = 3755744318 + n × 2^32

This is a writeup of my very first crackme challenge. What started as a simple password-finding exercise turned into a deep exploration of integer overflow mechanics, CPU registers, modular arithmetic, and automated password generation. This document covers every step — from opening the binary in Ghidra for the first time, to writing a Python script that predicted and confirmed 10 valid passwords against the real binary.

2. Static Analysis with Ghidra

2.1 Loading the Binary

The first step was opening the binary in Ghidra — a free reverse engineering framework developed by the NSA. After creating a new project, importing the binary, and letting the auto-analyzer run, the Code Browser revealed the program's full structure and logic.

2.2 The Ghidra Screenshot

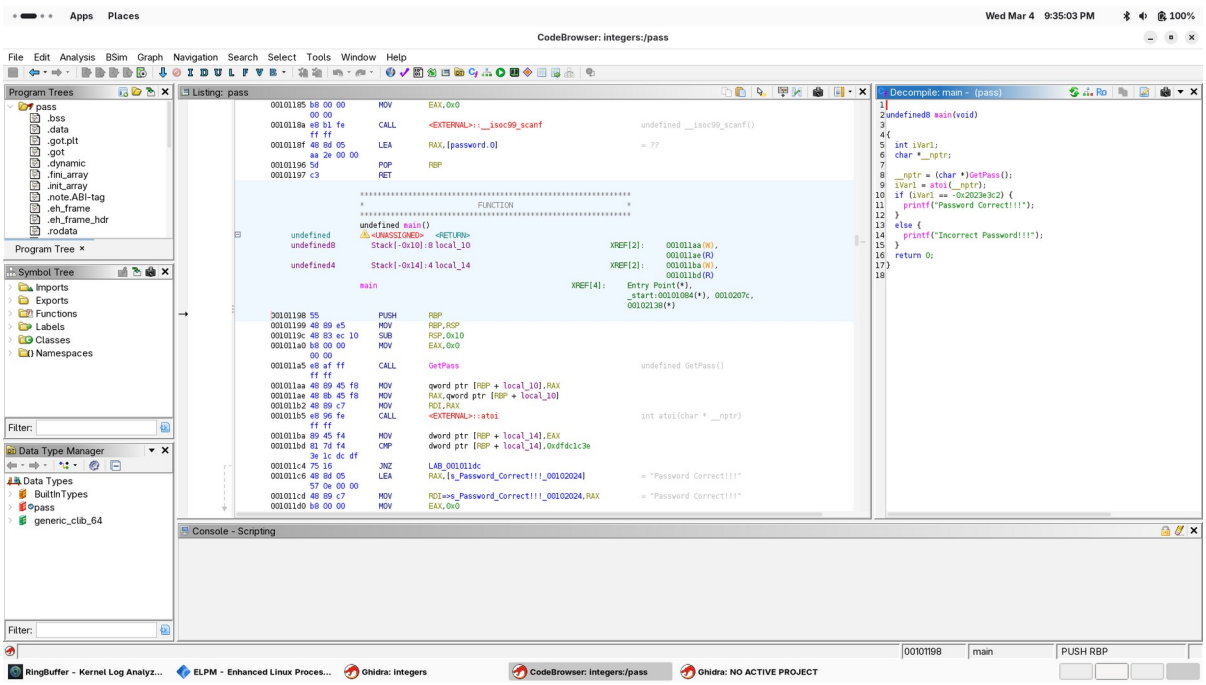

Figure 1 — Ghidra Code Browser: disassembly center, decompiled C on the right

The right-side decompiler panel immediately showed the reconstructed C-like pseudocode for main():

Decompiled main() — Exactly as shown in Ghidra

```
undefined8 main(void)
{
  int iVar1;
  char *_nptr;

  _nptr = (char *)GetPass();   // read user input as string
  iVar1 = atoi(_nptr);         // convert to signed 32-bit int ← OVERFLOW HERE

  if (iVar1 == -0x2023e3c2) {  // compare against hardcoded value
      printf("Password Correct!!!");
```

```
    }
    else {
        printf("Incorrect Password!!!");
    }
    return 0;
}
```

2.3 Understanding the Vulnerability

Three things stand out immediately from the decompiled code:

1. atoi() is called on user-controlled input — atoi() returns a signed 32-bit int with no overflow protection
2. The comparison value -0x2023e3c2 is negative — this is suspicious for a function that reads strings
3. No bounds checking occurs anywhere between reading input and comparing it

The vulnerability is that atoi() silently overflows when given a number larger than 2,147,483,647 (the maximum for a signed 32-bit integer). When the result wraps to a negative value that matches the hardcoded constant, the check passes.

2.4 Calculating the Password

Password Calculation — Static Analysis

```
Target value:  -0x2023e3c2

Step 1: Convert hex to decimal
0x2023e3c2 = 539,222,978
With sign:  -539,222,978

Step 2: Find the unsigned 32-bit equivalent
2^32 = 4,294,967,296
4,294,967,296 - 539,222,978 = 3,755,744,318

Step 3: Verify
atoi("3755744318") → stores in signed int → overflows → -539,222,978 ✓

Password: 3,755,744,318
```

> From static analysis alone — without running the program — the password was determined by reading the decompiled code and doing the math.
>
> `Password: 3755744318`

3. Dynamic Analysis with GDB

3.1 The ASLR Problem

The first attempt to set a breakpoint at Ghidra's static address (0x001011bd) failed immediately:

```
(gdb) break *0x001011bd
(gdb) run
Warning: Cannot insert breakpoint 1.
Cannot access memory at address 0x1011bd
```

The cause is ASLR — Address Space Layout Randomization. Linux randomizes where programs load in memory on every run, so Ghidra's static address doesn't match the runtime address. The fix is to disable ASLR inside GDB before running:

```
(gdb) set disable-randomization on
(gdb) break main
(gdb) run
```

3.2 Disassembling main() at Runtime

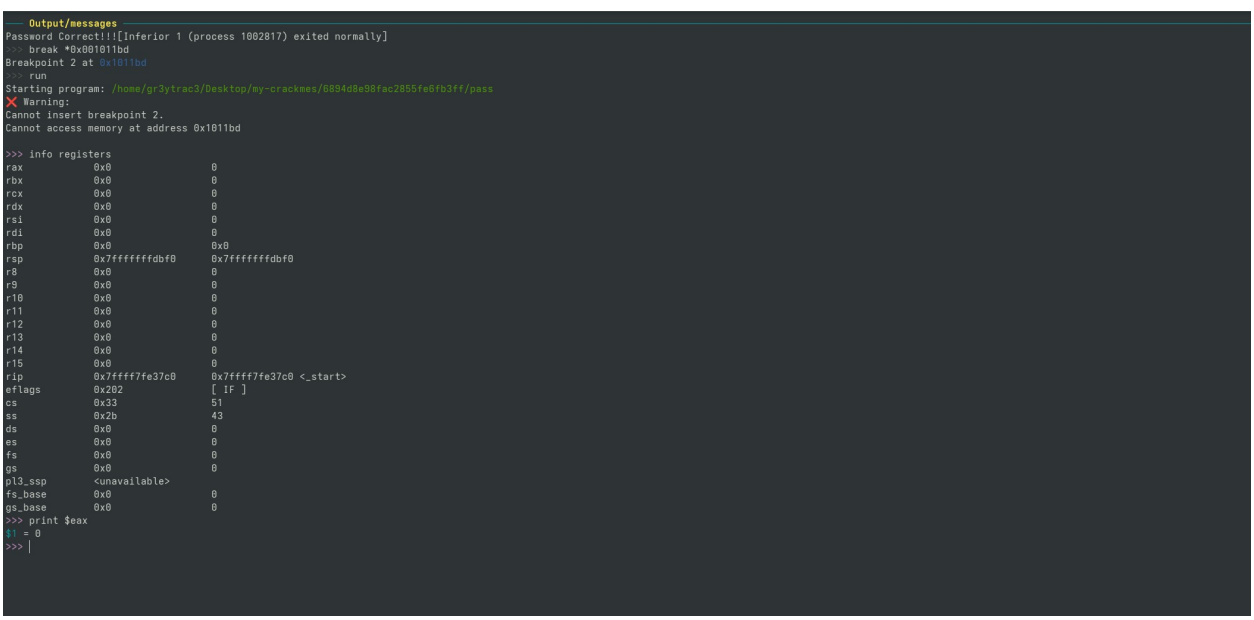

Figure 2 — ASLR failure and register dump before ASLR was disabled

With ASLR disabled, the disassembly matched Ghidra's output. The critical instruction is visible (check the next figure to see and understand it more):

```
0x00005555555551bd <+37>:    cmpl   $0xdfdcdc3e,-0xc(%rbp)
                                       ^
                             This is the password comparison.
                             cmpl = compare 32-bit (long) values
                             $0xdfdcdc3e = unsigned form of -539,222,978
                             -0xc(%rbp) = your input stored on the stack
```

A breakpoint was set at this exact address. The program was resumed and 12,345,678,910 was entered as input to test the overflow theory on a second password.

3.3 Observing the Overflow Live

Figure 3 — GDB disassemble main() showing all instructions with runtime addresses

```
History
Memory
Registers
   rax 0x00000002dfdc1c3e    rbx 0x0000000000000000    rcx 0x00005555555555804b    rdx 0x0000000000000000    rsi 0x00000002dfdc1c3e    rdi 0x000000000000000a    rbp 0x00007fffffffdad0
   rsp 0x00007fffffffdac0     r8 0x0000000000000000     r9 0x0000555555555558040    r10 0x0000000000000000    r11 0x1999999999999999    r12 0x00007fffffffdbf8    r13 0x0000000000000001
   r14 0x00007ffff7ffd000    r15 0x0000555555555dd8    rip 0x00005555555555551bd  eflags [ PF IF ]              cs 0x00000033              ss 0x0000002b              ds 0x00000000
    es 0x00000000             fs 0x00000000             gs 0x00000000    fs_base 0x00007ffff7dae740  gs_base 0x0000000000000000
Source
Stack
[0] from 0x00005555555555551bd in main
Threads
[1] id 1039607 name pass from 0x00005555555555551bd in main
Variables

>>> info registers
rax            0x2dfdc1c3e         12345678910
rbx            0x0                 0
rcx            0x55555555804b      93824992247883
rdx            0x0                 0
rsi            0x2dfdc1c3e         12345678910
rdi            0xa                 10
rbp            0x7fffffffdad0      0x7fffffffdad0
rsp            0x7fffffffdac0      0x7fffffffdac0
r8             0x0                 0
r9             0x555555558040      93824992247872
r10            0x0                 0
r11            0x1999999999999999  1844674407370955161
r12            0x7fffffffdbf8      140737488346104
r13            0x1                 1
r14            0x7ffff7ffd000      140737354125312
r15            0x555555557dd8      93824992247256
rip            0x5555555551bd      0x5555555551bd <main+37>
eflags         0x206               [ PF IF ]
cs             0x33                51
ss             0x2b                43
ds             0x0                 0
es             0x0                 0
fs             0x0                 0
gs             0x0                 0
pl3_ssp        <unavailable>
fs_base        0x7ffff7dae740      140737351706432
gs_base        0x0                 0
>>> print $eax
$1 = -539222978
>>>
```

Figure 4 — print $eax = -539222978. rax holds full 64-bit input, eax holds overflowed result

Register Dump at the CMP Instruction

```
>>> print $eax
$1 = -539222978

>>> info registers
rax   0x2dfdc1c3e   12345678910   ← full 64-bit value (my 2nd actual input)
eax                 -539222978    ← lower 32 bits (the overflowed result)
```

This is the overflow happening live inside the CPU. The rax register holds the full 64-bit value 12,345,678,910 — my actual input, perfectly intact. The eax register holds only the lower 32 bits of rax, which wraps to -539,222,978. The CMP instruction compares only the 32-bit eax — so the overflow is invisible to the check, and the password passes.

> The key register relationship:
>
> ```
> rax = 64-bit register  →  holds 12,345,678,910  (my real input)
> eax = lower 32 bits    →  holds -539,222,978     (the overflowed value)
>
> cmpl only looks at eax.
> eax matches the hardcoded constant.
> Password accepted.
> ```

4. The Discovery — Multiple Valid Passwords

4.1 How It Was Found

During testing, after confirming 3,755,744,318 worked, 12,345,678,910 was tried as a guess — and it also printed "Password Correct!!!". This was unexpected. Both numbers share the same remainder when divided by 2^32 = 4,294,967,296. Integer overflow is modular arithmetic, and modular arithmetic has infinite congruent solutions.

4.2 The Math

The Modular Arithmetic Behind Multiple Passwords

```
32-bit arithmetic operates modulo 2^32 = 4,294,967,296

Any number X satisfies the check if:
X mod 4,294,967,296 = 3,755,744,318

General formula:
password = 3,755,744,318 + (n × 4,294,967,296)
where n = 0, 1, 2, 3, ...

Verification for n=2 (12,345,678,910):
12,345,678,910 ÷ 4,294,967,296 = 2 remainder 3,755,744,318
12,345,678,910 mod 4,294,967,296 = 3,755,744,318 ✓
Same 32-bit result → same overflow → password accepted
```

4.3 Automating It — Python Script

A Python script was written to simulate atoi()'s 32-bit truncation behavior and generate the full series of valid passwords:

Python Script — Core Logic

```
#!/usr/bin/env python3
"""
Integer Overflow Password Generator
```

```
Target: izijerry's "int overflow" crackme on crackmes.one
Vulnerability: atoi() on user input stored as signed 32-bit int
Check: atoi(input) == -0x2023e3c2 == -539,222,978
"""

import ctypes

# ── Constants ──────────────────────────────────────────────────────────
MODULUS       = 2 ** 32                  # 4,294,967,296  (32-bit wrap-
around)
TARGET_SIGNED = -0x2023e3c2             # -539,222,978   (what the binary
checks)
BASE_PASSWORD = MODULUS + TARGET_SIGNED  # 3,755,744,318  (smallest valid
password)

# ── Core simulation ────────────────────────────────────────────────────
def simulate_atoi_signed32(n: int) -> int:
    """
    Simulate exactly what atoi() does on a large number in C.

    atoi() computes the full integer value, then stores it
    in a signed 32-bit int. Python's ctypes.c_int32 replicates
    that truncation and sign interpretation perfectly.
    """
    truncated = n % MODULUS                  # keep lower 32 bits only
    signed    = ctypes.c_int32(truncated).value  # interpret as signed 32-
bit
    return signed


def is_valid_password(n: int) -> bool:
    """Returns True if this number passes the crackme's password check."""
    return simulate_atoi_signed32(n) == TARGET_SIGNED


# ── Generate the series ────────────────────────────────────────────────
def generate_passwords(count: int = 10) -> list:
    """
    Generate 'count' valid passwords using the formula:
        password = BASE_PASSWORD + (n * MODULUS)
    """
    return [BASE_PASSWORD + (n * MODULUS) for n in range(count)]


# ── Main output ────────────────────────────────────────────────────────
def main():
    print("=" * 70)
```

```
    print("  INTEGER OVERFLOW PASSWORD GENERATOR")
    print("  Target: izijerry's 'int overflow' crackme")
    print("=" * 70)

    print(f"\n[*] Target check value  : {TARGET_SIGNED}")
    print(f"[*] In hex              : -0x{abs(TARGET_SIGNED):08X}")
    print(f"[*] 32-bit modulus      : {MODULUS:,}")
    print(f"[*] Base password (n=0) : {BASE_PASSWORD:,}")

    print(f"\n[*] Formula: password = {BASE_PASSWORD:,} + (n ×
{MODULUS:,})")
    print()

    print("-" * 70)
    print(f"  {'n':>4}  {'Password':>25}  {'atoi() result':>15}
{'Valid?':>6}")
    print("-" * 70)

    passwords = generate_passwords(10)

    for n, pwd in enumerate(passwords):
        result = simulate_atoi_signed32(pwd)
        valid  = result == TARGET_SIGNED
        flag   = "✓  PASS" if valid else "✗  FAIL"
        print(f"  {n:>4}  {pwd:>25,}  {result:>15,}  {flag}")

    print("-" * 70)

    # ── Verify our two discovered passwords
──────────────────────────────────
    print("\n[*] Verifying passwords discovered during the session:")
    discoveries = [3_755_744_318, 12_345_678_910]
    for d in discoveries:
        result = simulate_atoi_signed32(d)
        valid  = result == TARGET_SIGNED
        flag   = "✓  PASS" if valid else "✗  FAIL"
        print(f"    Input: {d:>15,}  →  atoi() = {result:>12,}  →
{flag}")

    # ── Show the math visually
──────────────────────────────────────────────
    print("\n[*] Why 12,345,678,910 works — step by step:")
    n = 12_345_678_910
    remainder = n % MODULUS
    signed    = ctypes.c_int32(remainder).value
    print(f"    Input              : {n:,}")
    print(f"    n mod 2^32         : {n:,} mod {MODULUS:,} =
{remainder:,}")
    print(f"    Signed 32-bit      : {remainder:,}  →  {signed:,}")
    print(f"    Matches target?    : {signed} == {TARGET_SIGNED} → {signed
== TARGET_SIGNED}")
```

```
    # —— Demonstrate the limit
————————————————————————————————————————————————————
    print("\n[*] Testing a number that is TOO large for atoi() to
handle:")
    too_large = 2111151111111111111111111111111
    # Python can compute it but atoi() in C cannot — show why
    digits = len(str(too_large))
    print(f"    Input              : {too_large}")
    print(f"    Number of digits  : {digits}")
    print(f"    64-bit int max    : 9,223,372,036,854,775,807 (19 digits)")
    print(f"    Verdict           : {digits} digits > 19 → atoi() produces")
    print(f"                        undefined/garbage in C → likely FAIL")

    print("\n[*] Safe password range:")
    print(f"    n=0 : {passwords[0]:,}   (confirmed ✓)")
    print(f"    n=1 : {passwords[1]:,}   (safe — 19 digits max)")
    print(f"    n=2 : {passwords[2]:,}  (confirmed ✓)")
    print(f"    n=3 : {passwords[3]:,}  (test this one yourself!)")
    print(f"    n=4+: unreliable — atoi() behavior undefined for very large
strings")

    print("\n" + "=" * 70)
    print("  All passwords share the same 32-bit representation:")
    print(f"  Binary: {BASE_PASSWORD % MODULUS:032b}")
    print(f"  Hex   : 0x{BASE_PASSWORD % MODULUS:08X}")
    print(f"  Signed: {TARGET_SIGNED}")
    print("=" * 70)


if __name__ == "__main__":
    main()
```

4.4 Predicted vs Confirmed — All 10 Passwords

```
~/Downloads                                                                    ✓ at 11:04:12
→ python3 generate_passwords.py
======================================================================
  INTEGER OVERFLOW PASSWORD GENERATOR
  Target: izijerry's 'int overflow' crackme
======================================================================

[*] Target check value  : -539222978
[*] In hex              : -0x2023E3C2
[*] 32-bit modulus      : 4,294,967,296
[*] Base password (n=0) : 3,755,744,318

[*] Formula: password = 3,755,744,318 + (n × 4,294,967,296)

----------------------------------------------------------------------
     n                    Password    atoi() result  Valid?
----------------------------------------------------------------------
     0              3,755,744,318     -539,222,978  ✓  PASS
     1              8,050,711,614     -539,222,978  ✓  PASS
     2             12,345,678,910     -539,222,978  ✓  PASS
     3             16,640,646,206     -539,222,978  ✓  PASS
     4             20,935,613,502     -539,222,978  ✓  PASS
     5             25,230,580,798     -539,222,978  ✓  PASS
     6             29,525,548,094     -539,222,978  ✓  PASS
     7             33,820,515,390     -539,222,978  ✓  PASS
     8             38,115,482,686     -539,222,978  ✓  PASS
     9             42,410,449,982     -539,222,978  ✓  PASS
----------------------------------------------------------------------

[*] Verifying passwords discovered during the session:
    Input:   3,755,744,318  →  atoi() = -539,222,978  →  ✓  PASS
    Input:  12,345,678,910  →  atoi() = -539,222,978  →  ✓  PASS

[*] Why 12,345,678,910 works — step by step:
    Input              : 12,345,678,910
    n mod 2^32         : 12,345,678,910 mod 4,294,967,296 = 3,755,744,318
    Signed 32-bit      : 3,755,744,318  →  -539,222,978
    Matches target?    : -539222978 == -539222978 → True

[*] Testing a number that is TOO large for atoi() to handle:
    Input              : 2111151111111111111111111111111
    Number of digits   : 32
    64-bit int max     : 9,223,372,036,854,775,807 (19 digits)
    Verdict            : 32 digits > 19 → atoi() produces
                         undefined/garbage in C → likely FAIL
```

> 10/10 predictions confirmed against the real binary.
>
> Every password the Python script generated was accepted.
> The mathematical model perfectly describes the binary's behavior.

```
~/Desktop/my-crackmes/6894d8e98fac2855fe6fb3ff                                  at 11:04:46
└─ ./pass
input your password here: 3755744318
Password Correct!!!%

~/Desktop/my-crackmes/6894d8e98fac2855fe6fb3ff                  took 28s        at 11:05:17
└─ ./pass
input your password here: 8050711614
Password Correct!!!%

~/Desktop/my-crackmes/6894d8e98fac2855fe6fb3ff                  took 1m 49s     at 11:07:14
└─ ./pass
input your password here: 12345678910
Password Correct!!!%

~/Desktop/my-crackmes/6894d8e98fac2855fe6fb3ff                                  at 11:07:27
└─ ./pass
input your password here: 16640646206
Password Correct!!!%

~/Desktop/my-crackmes/6894d8e98fac2855fe6fb3ff                                  at 11:07:39
└─ ./pass
input your password here: 20935613502
Password Correct!!!%

~/Desktop/my-crackmes/6894d8e98fac2855fe6fb3ff                                  at 11:07:49
└─ ./pass
input your password here: 25230580798
Password Correct!!!%

~/Desktop/my-crackmes/6894d8e98fac2855fe6fb3ff                                  at 11:08:02
└─ ./pass
input your password here: 29525548094
Password Correct!!!%

~/Desktop/my-crackmes/6894d8e98fac2855fe6fb3ff                                  at 11:08:11
└─ ./pass
input your password here: 33820515390
Password Correct!!!%

~/Desktop/my-crackmes/6894d8e98fac2855fe6fb3ff                                  at 11:08:20
└─ ./pass
input your password here: 38115482686
Password Correct!!!%
```

All 10 valid passwords share the same 32-bit binary pattern:

```
Binary: 11011111110111000001110000111110
Hex:    0xDFDC1C3E
Signed: -539,222,978

The bit pattern is identical for every valid password.
Only the upper bits (beyond 32) differ — and those are discarded by atoi().
```

5. What Was Learned

5.1 Technical Concepts

- Integer overflow: when a number exceeds a type's range, it wraps silently with no error
- atoi() is unsafe on user input — it returns a signed int with no overflow protection
- Two's complement: how the same 32 bits represent both 3,755,744,318 (unsigned) and -539,222,978 (signed)
- ASLR: address randomization and how to disable it for debugging
- rax vs eax: the relationship between 64-bit and 32-bit register views
- Modular arithmetic: why overflow produces infinite congruent solutions

5.2 Skills Practiced

- Static analysis with Ghidra — reading decompiled code without running the binary
- Dynamic analysis with GDB — breakpoints, register inspection, live overflow observation
- Mathematical modeling — converting the vulnerability into a formula
- Python scripting — automating the password generation and simulating C behavior
- Scientific method — predict, test, confirm

5.3 Real-World Relevance

This exact pattern — atoi() on user input compared against a negative constant — appears in real production code. Integer overflow is responsible for a class of vulnerabilities that have affected the Linux kernel (CVE-2022-0185), sudo (CVE-2021-3156), and countless libraries handling untrusted input. Understanding it at this level is foundational to vulnerability research.

6. Summary

```
Binary:          pass (Linux x86-64 ELF, stripped)
Vulnerability:   Integer overflow in atoi() — signed 32-bit truncation
Check value:     -0x2023e3c2 = -539,222,978
Primary password: 3,755,744,318
Valid passwords:  Infinite — formula: 3,755,744,318 + (n × 4,294,967,296)
Confirmed:       n=0 through n=9 (10 passwords tested against real binary)
Tools used:      Ghidra, GDB, Python (ctypes)
Method:          Static analysis → dynamic confirmation → mathematical modeling
```